

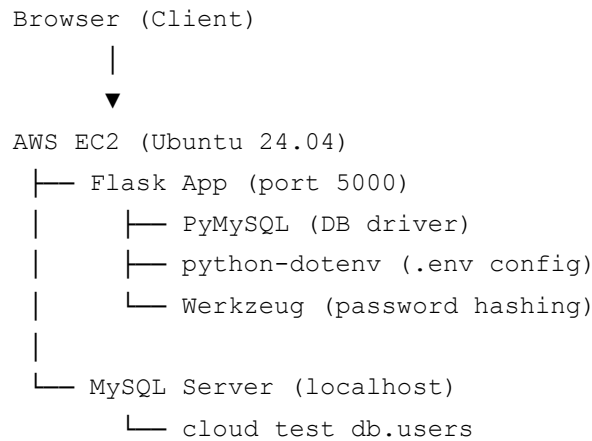
Multi-Tier Flask + MySQL Web Application

Cloud Computing & DevOps Practical Exam — a multi-tier web application demonstrating database connection pooling, secure authentication, and cloud deployment on AWS EC2.

Overview

This project simulates a real-world Cloud DevOps task: deploying a Flask web app backed by a MySQL relational database, with secure credential management, user authentication, and an admin monitoring portal — all live on an AWS EC2 instance.

Architecture



Features

- **User Registration** — new users sign up with hashed passwords (Werkzeug scrypt)
- **User Login** — session-based authentication
- **Protected Dashboard** — accessible only to authenticated users, shows profile details
- **Password Reset** — reset via username or email
- **Admin Portal** (`/admin/login`) — restricted to `role='admin'` accounts
- **Admin Dashboard** — live metrics (total users, total admins) + full registration table

Tech Stack

Component	Technology
Backend	Python 3, Flask
Database	MySQL 8.4
DB Driver	PyMySQL
Password Hashing	Werkzeug (bcrypt)
Config Management	python-dotenv
Deployment	AWS EC2 (Ubuntu 24.04)

Project Structure

```
flask-mysql-app/
├── app.py
├── requirements.txt
├── .env                # not committed - see .env.example
├── templates/
│   ├── register.html
│   ├── login.html
│   ├── dashboard.html
│   ├── reset_password.html
│   ├── admin_login.html
│   └── admin_dashboard.html
└── venv/              # not committed
```

Setup Instructions

1. Clone the repository

```
git clone https://github.com/Saahil5-byte/flask-mysql-app.git
cd flask-mysql-app
```

2. Set up MySQL database

```
CREATE DATABASE cloud_test_db;
```

```
USE cloud_test_db;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE,
  email VARCHAR(100) UNIQUE,
  password_hash VARCHAR(255),
  role ENUM('user', 'admin') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Create a dedicated app user (avoid using root):

```
CREATE USER 'flaskuser'@'localhost' IDENTIFIED BY 'your_password_here';
GRANT ALL PRIVILEGES ON cloud_test_db.* TO 'flaskuser'@'localhost';
FLUSH PRIVILEGES;
```

3. Set up Python environment

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Configure environment variables

Create a `.env` file in the project root:

```
MYSQL_HOST=localhost
MYSQL_USER=flaskuser
MYSQL_PASSWORD=your_password_here
MYSQL_DB=cloud_test_db
SECRET_KEY=your_secret_key_here
```

Never commit `.env` — add it to `.gitignore`.

5. Seed the admin account

```
python3 -c "from werkzeug.security import generate_password_hash; print(generate_
```

Copy the output hash, then:

```
INSERT INTO users (username, email, password_hash, role)
```

```
VALUES ('admin', 'admin@example.com', '<paste_hash_here>', 'admin');
```

6. Run the application

```
flask run --host=0.0.0.0 --port=5000
```

7. Open in browser

```
http://<your-ec2-public-ip>:5000
```

Cloud Deployment Notes

- Deployed on **AWS EC2** (Ubuntu 24.04, Free Tier eligible)
- App runs on `0.0.0.0:5000` to accept external traffic
- **Security Group** inbound rule added: Custom TCP, port `5000` , source `0.0.0.0/0`
- Verified live and accessible via public IP

Security Practices

- Passwords hashed with Werkzeug's `bcrypt` algorithm — never stored in plaintext
- Database credentials and secret keys loaded from `.env` , never hardcoded
- Role-based access control separates user and admin routes
- Dedicated MySQL user (`flaskuser`) with scoped privileges — app does not use `root`

Demo Credentials

Role	Username	Password
Admin	admin	admin123

(Register a new account via `/register` to test the standard user flow.)